



REINFORCEMENT LEARNING

AULA 4



Prof. Gian Carlo Brustolin



CONVERSA INICIAL

Conhecemos, anteriormente, duas formas de enfrentar problemas de aprendizagem em ambientes estocásticos. O método de Markov, equacionado por Bellman, tem certas condições de contorno restritivas, ao ponto que necessitamos de dados estatísticos sobre estados e ações para que possamos estimar a melhor política para cada estado.

O algoritmo de Monte Carlo fornece certa liberdade de predição mesmo em condições em que as relações estatísticas entre ações e estados, embora existam, não são conhecidas. Enfrentaremos esse método, com mais detalhes, na primeira parte desta etapa.

Ambos os métodos, programação dinâmica e Monte Carlo, entretanto, presumem um ambiente plenamente episódico. Dito de outra forma, os algoritmos evoluem bem se as decisões forem episódicas e markovianas de primeira ordem. Se o resultado de uma ação demorar a ocorrer e nova decisão precisar ser tomada pelo agente, antes de conhecer o resultado da ação anterior ou sua recompensa, os algoritmos falharão.

Podemos imaginar uma forma de prever o resultado de uma ação nos casos de ambientes estocásticos, mas não plenamente episódicos, nos quais seja necessário realizar períodos de decisões sequenciais antes de se conhecer o retorno efetivo do meio. Os métodos capazes de tratar esses problemas são ditos algoritmos de aprendizagem por diferença temporal ou simplesmente Temporal Difference (TD). Conheceremos esses algoritmos na segunda parte deste capítulo.

TEMA 1 – PREDIÇÃO POR MONTE CARLO

Assim como estudamos nos métodos de programação dinâmica, uma boa aproximação para que o agente seja capaz de agir inteligentemente em um ambiente desconhecido é a obtenção da função de valor de estado. Como o valor de um estado é composto pela somatória das expectativas de retorno a partir desse estado, podemos imaginar amostrar as recompensas após esse estado e tomar sua média, como uma forma de estimar o valor do estado atual. Recorrendo ao que aprendemos no exemplo do cálculo do valor de π , essa aproximação empírica é típica de algoritmos de Monte Carlo.



1.1 Modelos de Monte Carlo

Para que possamos saber quais recompensas amostrar, é necessário seguirmos ao menos uma política do agente. Desta forma, dois métodos de Monte Carlo (MC) são possíveis. O primeiro que estimará o valor do estado seguindo apenas os estados visitados após a primeira visita ao estado 's', seguindo uma política π , que denominaremos MC de visita única ou de primeira visita (First-Visit MC).

Podemos imaginar um segundo método que leve em conta todas as sequências a partir de todas as visitas possíveis a 's', esse método será dito MC de visita múltipla (Every-Visit MC). Como se quer uma estimativa computacionalmente ágil do valor do estado, First-Visit MC é mais eficiente e, por esse motivo, focaremos em seu estudo doravante.

Sutton e Barto (2018, p. 100) propõem o seguinte pseudocódigo para essa estimativa, supondo que mesmo para uma π qualquer podem existir mais de uma visita ao estado 's':

Início:

$\pi \leftarrow$ política a ser avaliada

$V \leftarrow$ função de valor de estado qualquer $s \in S$

$Returns(s) \leftarrow$ lista vazia, para todo $s \in S$

Loop:

Gerar episódio em π

Para cada estado s:

$G \leftarrow$ recompensas, seguindo a primeira ocorrência de s

Acrescente G to $Returns(s)$

$V(s) \leftarrow$ média ($Returns(s)$)

Observando o pseudocódigo, fica evidente que MC estima o valor de um estado 's' de forma, independentemente de qualquer outra estimativa de valor. Os métodos de DP necessitam envolver a estimativa de todos os valores de estado, o que torna a computação menos ótima, principalmente se temos um foco de estimativa de terminado.



1.2 Estimativa de Q

A aplicação do método de Monte Carlo, como vimos até agora, depende do conhecimento da política ou ao menos de uma política qualquer. De forma que possamos seguir a sequência de estados até o objetivo do agente, é necessário também conhecermos uma regra ou modelo de ação. Se esse modelo não estiver disponível, apenas os valores de cada estado não serão suficientes para avaliação da política. Uma aproximação possível se dá pela estimativa do valor de Q.

O problema do uso da estimativa de MC para o valor de Q é que MC é um método empírico baseado na estatística de ocorrência de um evento. Se temos uma política estabelecida π , em um estado 's', sempre tomaremos a ação 'a' determinada pela política, assim, não haverá variação estatística para permitir o uso de MC. Desta forma, a aplicação de MC só será possível para π estocástica, na qual exista a probabilidade não nula de ocorrer qualquer das ações possíveis em 's'.

Partindo do pressuposto estocástico anteriormente descrito, podemos utilizar o algoritmo *First-Visit* MC para avaliar π a partir de qualquer estado. É importante que se note que a estimativa do valor de um estado ou de Q se dá seguindo uma determinada política π , ou seja, $V(s)$ não será ótimo por essa aproximação, deveremos melhor controlar a escolha de π , o que veremos em seguida.

TEMA 2 – CONTROLE DA PREDIÇÃO POR ALGORITMO DE MONTE CARLO

Em nosso tópico anterior, entendemos como é possível avaliar o valor de um estado pelo algoritmo *First-Visit* MC e verificamos que essa estimativa não permite identificar a política ótima. O potencial de MC para otimização de política depende, como em DP, de um processo iterativo de ajuste mútuo da política e da função de valor de estado, isto é, tomamos uma política qualquer e calculamos a função de valor, a partir da função de valor, ajustamos a política, ajustada a política.

A otimização da função de valor pode ser feita por MC de duas formas genéricas, conforme as buscas são ou não direcionadas por uma política do agente. Desta forma, teremos métodos de controle que seguem a política (*on-*



policy) e métodos que não a seguem (*off-policy*). Para que iniciemos nosso estudo, devemos recordar algo sobre as estratégias de busca em árvores de decisão.

2.1 Busca informada gulosa

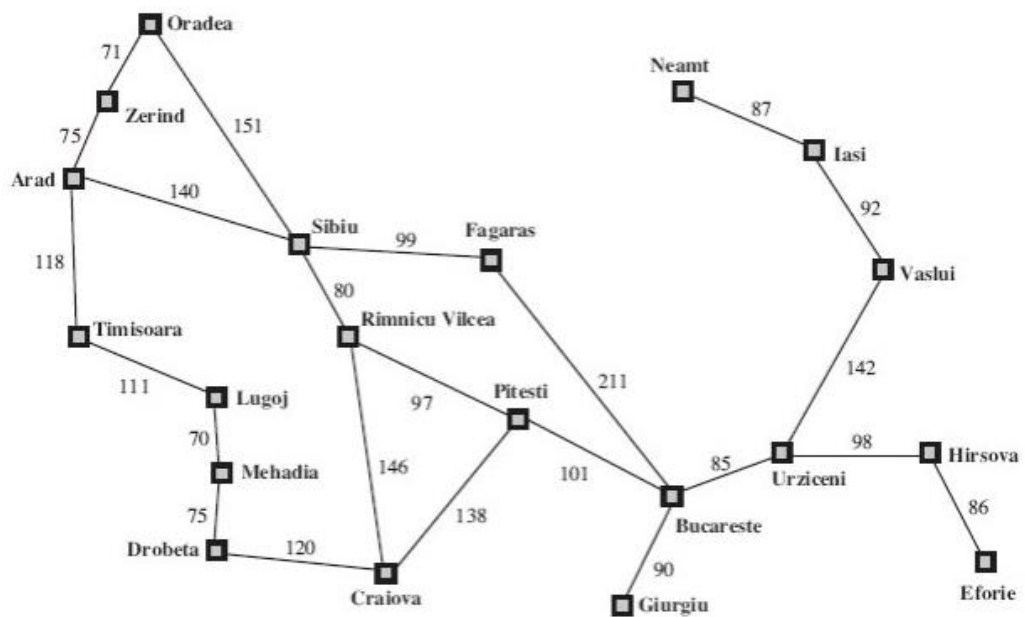
A aplicação do método de Monte Carlo *on-policy* para a aproximação da política ótima utiliza a metodologia de busca gulosa estudada nos fundamentos de IA. Vamos recordá-la com brevidade para que possamos dar sequência ao estudo de MC, recomendamos a aqueles que necessitem maiores detalhes a leitura da obra de IA de Norvig (2013, p. 82 e seguintes).

Em uma árvore de busca, podemos realizar a varredura para obtenção do resultado, segundo dois tipos genéricos de estratégias. A busca pode ser realizada segundo um algoritmo “cego”, ou seja, que não possua informações que possibilitem direcionar a busca, a esses algoritmos denominamos algoritmos de busca cega. Por outro lado, podemos ter algum tipo de informação que nos ajude a selecionar algum caminho preferencial de busca. Essas informações são ditas *heurísticas* e há alguns métodos para que geremos heurísticas para uma árvore de busca. A essas estratégias de busca denominamos de *buscas informadas*.

Para esclarecermos o que dissemos, vamos usar o exemplo dado por Norvig (2013), que se refere ao problema clássico do caixeiro viajante. Um vendedor deve encontrar a melhor rota entre duas cidades romenas, Arad e Eforie. Várias cidades estão no caminho e várias rodovias estão disponíveis.



Figura 1 – Mapa rodoviário romeno proposto

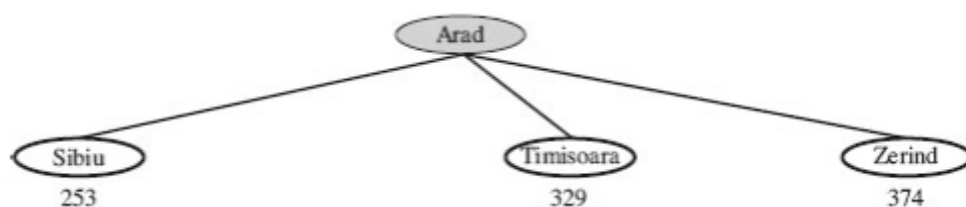


Fonte: Norvig, 2013, p. 62.

Encontrar a solução ótima depende de algum algoritmo de busca. Para direcionar o algoritmo, podemos criar uma heurística, $h(t)$. A sugestão do autor é a distância em linha reta, ou seja, conhecendo a distância aérea entre Arad e Eforie, podemos julgar se o caminho que estamos percorrendo tende a ser ótimo ou não. Bastaria comparar a soma das distâncias percorridas com a heurística. Aquela busca que mais se aproximar de $h(t)$ será a melhor escolha de roteiro.

Ainda assim, a busca precisa seguir uma dada ordem. Se tomarmos a árvore de busca para esse problema, por onde começamos para que comparemos com a heurística?

Figura 2 – Árvore de busca



Fonte: Norvig, 2013, p. 69.

Na figura anterior, criamos a árvore de busca a partir de Arad, com as três possibilidades de trajetos, mas e agora? Qual a próxima expansão da árvore?



Uma das estratégias tradicionais de busca informada é a dita busca gulosa (*Greedy Search*). Os algoritmos de busca gulosa expandem sempre o nó mais próximo ao destino. Para avaliarmos qual o nó nesta situação, na busca gulosa, vamos expandir o conceito da heurística, antes mencionada para uma nova $h(t)$, que calcula a distância aérea de cada novo nó, expandido em relação ao destino.

Suponha que essas distâncias sejam aquelas que estão abaixo dos nós da figura anterior. Com o uso da nova $h(t)$ podemos escolher Sibiu. A partir de Sibiu, expandimos os nós filhos e aplicamos $h(t)$ para cada um deles para facultar a decisão pelo “melhor” filho.

Note que a busca gulosa não é completa, ou seja, pode não chegar ao objetivo. Isto ocorrerá se for necessário, para a melhor rota, em algum momento escolher temporariamente o pior caminho entre cidades. Nestes casos, a busca gulosa falhará.

Para contornar esse problema de completitude, podemos imaginar uma escolha estocástica, ou seja, no exemplo de Norvig anteriormente, podemos pensar que $h(t)$ poderia ser combinada a uma probabilidade de escolha. Assim, mesmo por busca gulosa, poderíamos escolher um filho “pior” com probabilidade ϵ . Essa modificação é chamada de busca ϵ -gulosa, em que a escolha de determinada ação, por maior valor que tenha, depende da probabilidade ϵ .

2.2 Controle de MC *on-Policy*

Já entendemos o conceito de busca gulosa, então podemos seguir com a definição de como o algoritmo de controle de Monte Carlo *on-policy* atua. A busca utiliza estratégias ϵ -gulosas, em que ϵ é a probabilidade não se escolher a ação de maior Q .

Neste caso, supondo de $A(s)$, identifique o conjunto de todas as ações não ótimas, com isso, podemos afirmar que uma ação não gulosa tem probabilidade calculada como $\epsilon / n(A(s))$ para todo ‘s’ (Sutton; Barto, 2018, p. 127). Seguindo o proposto pelo mesmo autor, podemos generalizar a estratégia de controle pelo pseudocódigo a seguir:

Início:

$\Pi(a|s) \leftarrow$ política qualquer tendente a ϵ -gulosa



```
Q (s,a) ← função Q em estado qualquer s ∈ S e a ∈ A
Returns (s, a) ← lista vazia
Loop {
  Gerar episódio em π
  Para cada s, a: {
    G ← recompensas, seguindo a primeira ocorrência de s,a
    Acrescente G to Returns(s,a)
    Q (s,a) ← média>Returns(s,a))
  }
  Para cada s: {
    A* ← arg maxa Q(s, a)
    Para todo a ∈ A(s): {
      π(a|s) ← 1 - ε + ε/|A(s)|   se a = A*
      π(a|s) ← ε/|A(s)|           se a ≠ A*
    }
  }
}
```

Os algoritmos de controle *on-policy* são normalmente mais simples e de convergência rápida, em relação a seus irmãos *off-policy*. Por esse motivo, são sempre a primeira opção a considerar.

2.3 Controle de MC *off-Policy*

Já entendemos como o algoritmo de controle de Monte Carlo *on-policy* age, mas, neste modelo estudado, para encontrar a política ótima, é necessário realizar sondagens não ótimas. Esse paradigma torna impossível encontrar efetivamente a política ótima. Encontraremos, entretanto, a política tanto mais próxima da ótima quanto quisermos. Os algoritmos *off-policy* resolvem esse problema de tangenciamento infinito pela proposição de duas políticas simultâneas. Uma política exploratória, como no caso anterior, e uma política iterativa, que se tornará ótima.

Desta forma, em um algoritmo *off-policy*, deveremos lidar com duas políticas o que se reflete em tempo computacional e maior variância das iterações em relação ao objetivo. Essa técnica mais complexa, todavia, permite a solução de alguns problemas impossíveis para a aproximação anterior.



Segundo Sutton e Barto (2018, p. 111), controles *off-policy* permitiriam ao algoritmo de Monte Carlo tratar problemas de decisão não episódica, inacessíveis para a abordagem *on-policy*, uma vez que o algoritmo não é capaz de tratar um processo de ordem superior de Markov.

Neste estudo, não abordaremos o controle *off-policy*, uma vez que sistemas não episódicos podem ser melhor equacionados por Diferença Temporal, tema que aprofundaremos a seguir. Deixamos ao leitor interessado em conhecer o controle fora de política a recomendação pela leitura da obra de Sutton e Barto (2018, p. 112-116).

TEMA 3 – INTRODUÇÃO À DIFERENÇA TEMPORAL (TD)

A aprendizagem por diferença temporal (TD do inglês *Time Difference*) é, de fato, um conjunto de algoritmos central do aprendizado por reforço. Algoritmos de TD permitem uma aproximação mais genérica do problema de aprendizagem em ambientes mutáveis.

Em TD, a função de valor $V(s)$ é calculada diretamente a partir do erro de previsões anteriores, livre de modelo, como em Monte Carlo, de forma completamente incremental, mas sem a restrição do processo à primeira ordem da cadeia de Markov. Por tal motivo, parte da literatura considera TD como uma associação dos pontos positivos de Bellman e Monte Carlo (Lins, 2020). Neste tópico, entenderemos como isto é possível e conhecer a aplicação mais “popular” desses algoritmos: a Q-learning.

Aprendemos que, quando em um MDP não conhecemos as probabilidades de transição e de recompensas, o algoritmo de Monte Carlo é uma solução viável. Esse algoritmo enfrenta o problema por amostragens e aproxima o valor das distribuições de probabilidade de estados, ações e recompensas, tornando a solução do MDP possível. Do ponto de vista de implementação matemática, a solução de Monte Carlo é mais simples que a aproximação pela programação dinâmica, para a determinação da política ótima. Mas e se o retorno de meio não for imediato?

O algoritmo de Monte Carlo depende de conhecermos o espaço de entorno markoviano, ou seja, de conhecermos pais e filhos do estado. Se não conhecermos a reação do meio, não saberemos que filhos explorar. Algoritmos de TD libertam-se, ao menos parcialmente, dessa dependência de entorno.



Escolhem uma aproximação temporal e não a tradicional exploração sequencial de estados segundo a política, que vincula o meio à característica episódica.

3.1 Visão histórica

O engenheiro Arthur Samuel foi o primeiro cientista da computação a propor algoritmos eficientes de aprendizado de máquina (no sentido lato). Samuel, além de formular matematicamente os problemas, testou com sucesso seus algoritmos em implementações práticas de jogos inteligentes. Essas implementações, que datam da metade do século passado, deram credibilidade ao estudo da IA, que, até aquele momento, era majoritariamente encarada como uma ciência não pragmática.

O aporte de inteligência em jogos complexos, que, justamente por sua complexidade inerente, relativizam a importância de um aprendizado prévio, por métodos simbólicos, impulsionaram a criação de métodos de aprendizagem adaptativa.

A previsibilidade da melhor jogada (política ótima) pela resolução das equações de Bellman, para cada movimento do adversário, tornavam, entretanto, o processo de cálculo lento para a capacidade computacional da época. Desse desafio surgiu a ideia da aprendizagem temporal de Samuel que foi aperfeiçoada por Richard Sutton, nas décadas seguintes, ao ponto de permitir a criação de agentes com nível de inteligência indiscriminável em relação aos jogadores humanos.

A leitura do artigo de Sutton (1988, p. 9-44), *“Learning to Predict by the Methods of Temporal Differences”*, que será resumido nos próximos tópicos, nos fornece uma visão bastante sólida de TD e é bastante indicada para aqueles que se dedicarem às implementações de RL.

3.2 Predição em MDPs de Ordem Superior

Os métodos tradicionais de aprendizagem que abordamos superficialmente anteriormente, a exemplo da regra de Windrow-Hoff e da aprendizagem por memorização, são facultados pelo controle do erro entre a predição e a saída atual, em ambientes determinísticos. Na solução de problemas não determinísticos, como em DP, estudada anteriormente, a



aproximação segue semelhante, comparando-se a predição com a reação atual do meio. TD busca a previsão da ação baseada na análise de erro das sucessivas predições temporais já feitas. O aprendizado ocorre toda vez que há uma mudança no tempo da predição. Vamos seguir o exemplo proposto por Sutton (1988, p. 10) para elucidar o que afirmamos anteriormente.

Imagine que se deseje fazer uma previsão meteorológica sobre a condição do tempo em um dado sábado. Nos algoritmos tradicionais, tomaríamos o estado marcoviano, portanto, de sexta-feira, para fazer a previsão. Se na sexta-feira chove ou não, isso influenciará diretamente na previsão de sábado e apenas de sábado.

Pela aproximação de TD, comparamos as previsões dos dias anteriores e suas relações para prever o dia posterior. Se o algoritmo previu 50% de chance de chuva na segunda-feira e 75% na terça-feira e obteve êxito, então devemos aumentar as probabilidades de chuva para dias semelhantes à terça-feira em dias semelhantes a segunda. Se nesta sexta-feira, a previsão de chuva era de 50%, podemos aumentar a probabilidade de sábado chover, para além de 75%, segundo essa aproximação.

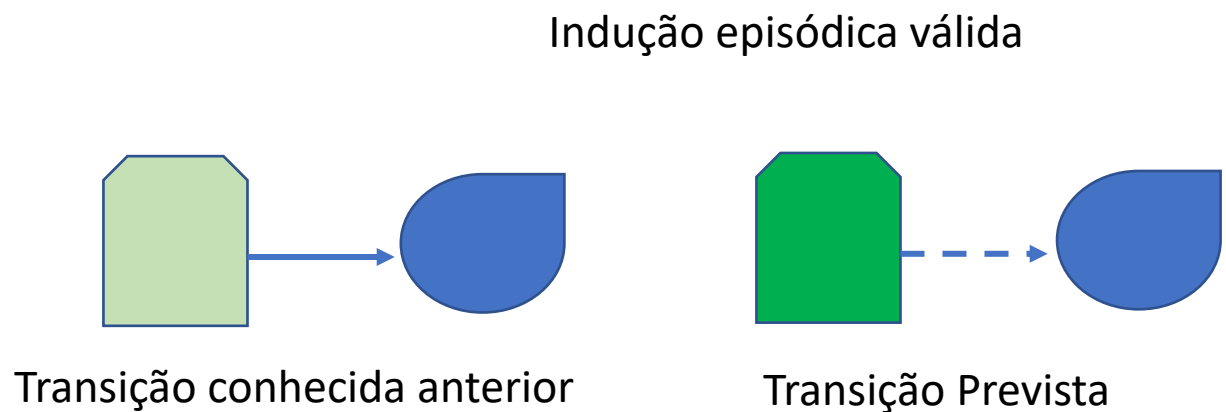
O algoritmo poderá também incrementar a acuidade dos resultados de todas as previsões de tempo após analisar o resultado de sábado, enquanto algoritmos marcovianos clássicos aguardarão o próximo evento similar, isto é, o próximo sábado, para corrigir as previsões de sábados apenas.

Essa aproximação demandará menor capacidade de memória e usará melhor as experiências anteriores. Segundo Sutton, isto tornará os problemas mais simples e eficientes do ponto de vista computacional.

Sutton (1988, p. 10) define duas formas diferentes de encarar um problema de predição, a predição de passo único e a de passo múltiplo. A figura a seguir nos ajudará a compreender a ambas.



Figura 3 – Previsão em meios episódicos

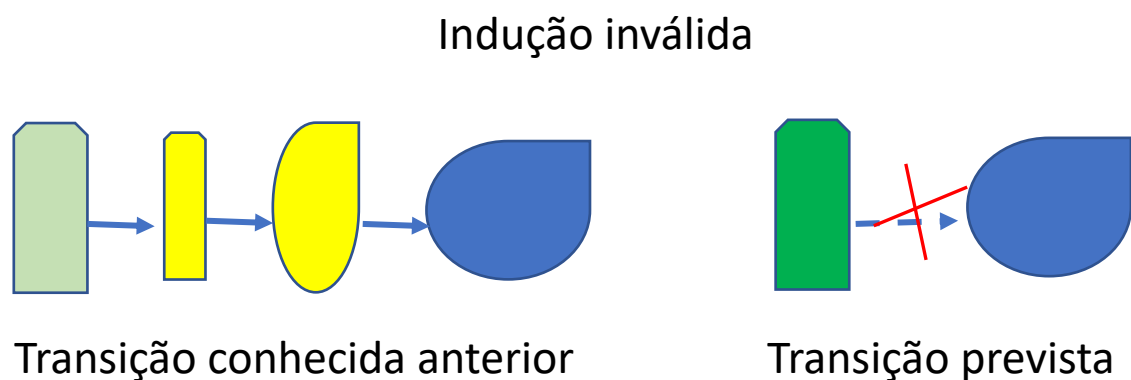


Neste caso, a previsão está baseada em um único par (X, y) . Podemos retomar o treinamento neural pelo algoritmo *Backpropagation*, de correção de erro, visto anteriormente, como exemplo. Neste algoritmo, a cada vetor de entrada X , a rede neural é treinada a retornar um valor de saída y semelhante a 'd', ou seja, ajustamos os parâmetros livres da rede em função da saída desejada 'd', conhecida do algoritmo antes do início do treinamento.

Desta forma, nesse treinamento neural, um estado 'X' transitará para o estado 'y' baseado apenas no conhecimento anterior que temos das transições bem sucedidas de 'X' para 'y'. Assim, trata-se de transição de passo único, ou episódica, pois dependemos de uma única informação para prever a transição.

Transições de múltiplos passos dependem de uma sequência de informações para que a decisão seja eficiente, como se vê na figura a seguir.

Figura 4 – Previsão em meios não episódicos





No exemplo do meteorologista, uma boa previsão para sábado pode ser obtida se analisarmos uma sequência similar de dias que antecedem sábado. A TD é uma aproximação eficiente para esse segundo caso.

Dito de outra forma, o cálculo do valor de uma ação 'a' em um estado "s" pode ser feito pela soma da recompensa imediata da transição de "s" para "s'" com a somatória das probabilidades anteriores de transição para "s'" a partir de "s" pela ação "a", representada por $T(s'|s, a)$, multiplicada pelo valor do estado "s'", seguida a mesma política π . Essa aproximação permite levar em conta as decisões passadas para valorar a ação $Q(s, a)$.

O equacionamento dessa afirmação pode ser feita como a seguir (Pellegrini; Wainer, 2007, p. 139):



$$Q^{\pi}(s, a) = R(s, a) + \gamma \sum_{s' \in S} T(s'|s, a) V^{\pi}(s').$$

Neste ponto, devemos estar seguros da distinção objetiva entre V^{π} e Q^{π} . A função que define o valor do estado V^{π} nos informa quão bom é esse estado, ao passo que Q^{π} mede quão boa é uma determinada ação nesse estado. A notação V^{π} e Q^{π} indicam que esses valores são obtidos se seguida a política π .

Seguindo a mesma ideia da aloração dos estados anteriores, podemos escrever a equação que define o valor de um estado como (Pellegrini; Wainer, 2007, p.138):

$$V_k^{\pi}(s) = R(s, d_k(s)) + \gamma \sum_{s' \in S} T(s, d_k(s), s') V_{k+1}^{\pi}(s')$$

Observe que essas equações muito se assemelham às que usamos até o momento, mas aqui a ênfase à probabilidade de transição é expressa.

3.3 A escolha do estado ruim

Quando conceituamos a função de valor de um estado, V^{π} dissemos que se, para o mesmo estado $V^{\pi_1} > V^{\pi_2}$, então a política π_1 é melhor que a política π_2 , já que;

$$V^*(s) = \max_{\pi} V^{\pi}(s)$$

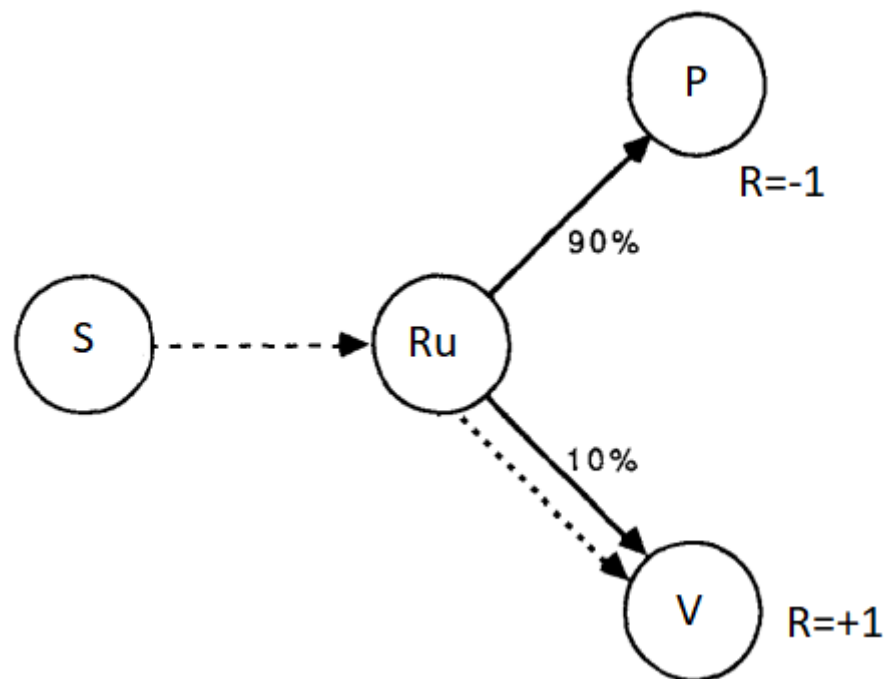
Essa conclusão até o momento nos parecia evidente, mas Sutton nos propõe uma decisão interessante em um jogo. Suponha que você tenha a distribuição de probabilidade descrita na figura a seguir. Neste caso, o valor do estado 'Ru' é baixo, pois existe uma probabilidade de 0,9 de perdermos a recompensa se estivermos em 'Ru', transitando para 'P'. O algoritmo não escolheria transitar do estado 'S' para 'Ru', uma vez que a segunda conclusão de Bellman estabelece que:

$$V^*(s) = \max_a Q^*(s, a)$$



Impedindo a escolha de 'Ru' por não ser ótima.

Figura 3 – Escolha do estado Ruim



Fonte: adaptado de Sutton, 1989, p. 17.

Por outro lado, existe uma probabilidade de 10% de se obter a recompensa, do estado V. Imagine que, dependendo da sequência de jogadas anteriores, a única oportunidade de vitória se dará por ir ao estado 'Ru', que, neste caso, tem uma probabilidade de sucesso real, maior que 10%. O algoritmo tradicional de programação dinâmica não escolherá 'Ru' como uma saída viável e falhará, mas a técnica de TD é capaz de atualizar a probabilidade de 'Ru' ir para 'V' obtendo sucesso.

De fato, essa conclusão não invalida a solução de Bellman, mas apenas acrescenta a necessidade de se considerar, em determinados casos, um MDP de ordem mais elevada. Sutton conclui (1989, p. 29), por outro lado, que mesmo em processos marcovianos de primeira ordem o uso de TD é convergente e ao menos igualmente eficiente que outros métodos de aprendizagem, o que transforma TD em um segundo método de solução da equação de otimalidade de Bellman.



TEMA 4 – PREDIÇÃO POR DIFERENÇA TEMPORAL

De forma a operacionalizar os conceitos de aprendizado por TD, devemos entender como a função de valor $V(s)$ pode ser dinamicamente calculada. Como adiantamos, no tópico antecedente, o método passa pela análise dos erros das previsões anteriores, ao qual denominaremos erro por diferença temporal, ou erro TD.

Calculado o erro TD, pode-se atualizar o valor de $V(s)$ de forma similar ao que se faz em redes neurais treinadas pela regra de Windrow-Hoff. Entenderemos melhor a mecânica dessa técnica de treinamento neural em nosso próximo capítulo, por hora basta-nos compreender que podemos ajustar um parâmetro qualquer iterativamente pela soma de seu valor original com o ajuste em relação ao valor desejado. Tomando $V(s)$, poderíamos escrever então:

$$V(s_{\text{ajustado}}) = V(s_{\text{original}}) + \text{erroTD}$$

4.1 Erro TD

O erro por diferença temporal, como já adiantamos, é o fator que ajustará o valor do estado em função das ocorrências anteriores que contribuem para o próximo estado. Anteriormente, conceituamos a utilidade em função das recompensas em processos marcovianos de ordem 'n' como sendo:

$$U_h([s_0, s_1, s_2, \dots]) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots,$$

Nesta aproximação, γ representa um fator de apreciação da atualidade da recompensa. Podemos julgar a assertividade de uma ação anterior pela recompensa 'R' por ela obtida. Em um meio episódico, poderíamos escrever que o valor de um estado 's' é igual à soma do valor do estado futuro ' s_{t+1} ' somado à recompensa obtida nesse estado ' R_{t+1} '. Assim, para o meio episódico:

$$V(S_t) \doteq R_{t+1} + \gamma V(S_{t+1})$$

Entretanto, como não há determinação absoluta do estado futuro pelo estado atual, a previsão de $V(S)$ pela equação anterior sofrerá um erro que



chamaremos de δ . Com isso, escreveremos o valor seguindo as conclusões de Sutton e Barto (2018, p. 129):

$$\delta_t \doteq R_{t+1} + \gamma V(S_{t+1}) - V(S_t).$$

Desta forma, o erro por diferença temporal será a diferença do valor efetivo do estado futuro e do valor estimado para esse estado. Assim, o erro TD só será mensurável quando atingirmos o tempo (t+1), ou seja, o erro δ_t do valor do estado $V(S_t)$ só será calculável em (t+1).

4.2 Atualização do Valor do Estado

Considerando-se o que estudamos anteriormente, podemos agora equacionar o valor do estado futuro a partir da correção de sua estimativa pelo erro da diferença temporal. Nesta equação, por ser um processo iterativo, é interessante descontarmos o valor do erro por uma taxa, que denominaremos taxa de velocidade de aprendizagem, ou simplesmente taxa de aprendizagem α .

A utilidade de α será melhor discutida a seguir, mas adiantamos que processos iterativos tendem a divergir se utilizarmos passos muito abruptos, assim, α auxiliará na suavização das iterações. De qualquer forma, o novo valor de $V(S)$ pode então ser corrigido como segue (Lins, 2020, p. 34):

$$V(s) \leftarrow V(s) + \alpha[R_t + \gamma V(s_{t+1}) - V(s)],$$

Voltaremos ao ambiente do lago congelado para exemplificar a atualização de $V(s)$ seguindo o ensinado por Ravichandiran (2018, p. 92). Assumirmos a configuração default abaixo e consideraremos todos os valores de estado iniciais iguais a zero, como fizemos nos processos de iteração de valor por DP.

Figura 4 – Disposição clássica do ambiente Frozen Lake

S	F	F	F
F	H	F	H
F	F	F	H
H	F	F	G



Estando na posição S, se nos movimentarmos à direita, iremos ao estado determinado pela posição matricial (1,2), supondo que recebamos uma recompensa $R=-0,3$, podemos calcular um novo valor do estado inicial $V(S)$. Se escolhermos $\alpha=0,1$ e $\gamma=0,5$ teremos todos os dados para recalcular $V(S)$ pela equação que vimos anteriormente:

$$V(S) = 0 + 0,1(-0,3 + 0,5 \cdot 0) = -0,03.$$

Desta forma, atualizamos o valor do estado S ou, matricialmente (1,1), $V(1,1) = -0,03$. Supondo agora um novo movimento para a direita, a partir de (1,2), nos levando a (1,3), teremos, se a recompensa for também -0,3, $V(1,2) = -0,03$. Até este ponto, não há novidade, mas vamos agora, a partir do estado em que estamos (1,3), retornar a (1,2) por um movimento à esquerda.

Neste caso, quando aplicarmos a formulação proposta por Sutton, teremos uma mudança, vejamos: o valor do estado (1,3), $V(1,3) = 0$, uma vez que todos os estados foram inicializados em zero, mas o estado futuro (1,2) já possui um valor anterior, que calculamos acima como $V(1,2) = -0,03$. Então na atualização de $V(1,3)$ que era zero, teremos:

$$V(1,3)_{\text{novo}} = V(1,3) + \alpha \{ R(1,2) + \gamma V(1,2) - V(1,3) \} = 0 + 0,1 \cdot (-0,3 + 0,5 \cdot (-0,03) - 0) \\ V(1,3)_{\text{novo}} = -0,0315.$$

Esse procedimento pode ser repetido iterativamente até completarmos os valores de todos os estados.

TEMA 5 – CONTROLE DO ALGORITMO DE APRENDIZAGEM POR TD

Nos tópicos que estudamos até o momento, entendemos como atualizar iterativamente os valores de cada estado, mas devemos otimizar esse processo de forma a rapidamente encontrarmos a função de valor ótima para o problema.

5.1 Q-Learning

Bastante conhecidos por sua utilização em processos de aprendizagem por reforço profunda em MLPs, os algoritmos de aprendizagem Q são métodos de controle de convergência da aprendizagem por diferença temporal em direção à função ótima de valor.



Como se pode suspeitar, pela nomenclatura utilizada, esse algoritmo maximiza o valor da função de estado indiretamente, pela maximização do valor das ações 'a' nos diferentes estados 's'. Como as conclusões de Bellman nos permitem a substituição de $V(s)$ por $Q(s)$ na equação iterativa de Sutton, podemos escrever:

$$Q(s, a) = Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s' a') - Q(s, a))$$

Desta forma, será possível comparar duas ações e decidir por aquela que tem maior valor de Q , ou ($\max Q$), e só então atualizar o valor do estado para essa melhor ação. No exemplo que usamos anteriormente, quando estávamos no estado (1,3) decidimos, aleatoriamente, voltar para (1,2) e alteramos o valor de $V(1,3)$ a partir dessa decisão. Mas ir para a esquerda em $V(1,3)$ é a melhor ação?

Na verdade, quando tomamos essa decisão, não o sabíamos, mas o cálculo de Q nos conduzirá à melhor escolha, dirigindo o algoritmo de TD em direção ao valor ótimo e, conseqüentemente, à política ótima. A busca se dá como em uma estratégia ϵ -gulosa que encontra o próximo estado, segundo a função heurística, selecionando uma ação aleatória qualquer com probabilidade ϵ e a ação que retorna o maior valor heurístico com probabilidade $(1 - \epsilon)$.

Utilizamos Q como heurística de escolha ao mesmo tempo que atualizamos Q . Isto é possível porque a atualização é sempre feita em $(t+1)$, como preconizado por Sutton.

Vamos novamente voltar ao ambiente do lago congelado para exemplificar a atualização de $Q(s)$, seguindo o exemplo proposto por Ravichandiran (2018, p. 96).

Se estivermos em (3,2) e os valores das ações esquerda, direita e abaixo forem respectivamente 0,1; 0,5 e 0,8, tomaremos a decisão de descer, atingindo o estado (4,2). A decisão é tomada baseada na estratégia de busca ϵ -gulosa. Neste ponto, o algoritmo pode escolher seguir a ação de maior Q , isto é, de probabilidade $(1 - \epsilon)$, ou qualquer das outras ações, de Q menor, e, portanto, probabilidade ϵ . Supondo a escolha $(1 - \epsilon)$, descenderemos para (4,2).



Agora que estamos no novo estado, pela regra de TD, estabelecida por Sutton, podemos atualizar o valor da ação no estado anterior, $Q(3,2)$ para a ação “descer”, que denominaremos $Q(3,2)d$. A equação a ser resolvida é:

$$Q(s, a) = Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

Para equacionarmos $Q(s, a)$, precisaremos conhecer a taxa de aprendizagem α e o desconto temporal γ . Ambos os valores são uma escolha de cálculo e podem ser ajustados para otimizar a convergência. Vamos considerar $\alpha = 0,1$ e $\gamma = 1$. Lembremos que, no ambiente Frozen Lake, a probabilidade de transição é de 0,3, como comentado anteriormente. Feita essa escolha de parâmetros, calcularemos $Q(3,2)d$ pela equação anterior.

$$Q(3,2)d_{\text{novo}} = Q(3,2)d + 0,1(0,3 + 1 * \max Q(4,2) - Q(3,2)d)$$

O valor de $Q(3,2)d$ foi suposto em 0,8 algumas linhas acima. Quanto ao valor de $\max Q(4,2)$, dependemos de conhecer a tabela dos valores de Q anteriores (como no caso de $Q(3,2)$). Conhecida essa tabela, escolhemos o valor máximo. Apenas para seguir o exemplo proposto por Ravichandiran, suponha que os valores das ações esquerda, direita e abaixo, em $Q(4,2)$ sejam respectivamente 0,3; 0,5 e 0,8. Desta forma, $\max Q(4,2)$ será também 0,8. Finalmente:

$$Q(3,2)d_{\text{novo}} = Q(3,2)d + 0,1(0,3 + 1 * \max Q(4,2) - Q(3,2)d) = 0,8 + 0,1 * (0,3 + 0,8 - 0,8)$$

$$Q(3,2)d_{\text{novo}} = 0,83$$

Estamos agora no estado $Q(4,2)$ e podemos escolher um novo movimento exploratório para seguir atualizando a tabela de Q s. Esse processo então se repete até que alcancemos o estado final (G).

Neste exemplo, partimos de uma tabela Q já conhecida, mas não há impedimento de partirmos de valores aleatórios. O pseudocódigo a seguir, proposto por Lins (2020, p. 34), ilustra essa possibilidade.



```
início
| Inicialize  $Q(s, a)$  aleatoriamente
| repita
| | Inicialize  $s$ 
| | repita
| | | Selecione  $a$  a partir de  $s$  utilizando  $\pi$  derivada de  $Q$  (por ex.  $\epsilon$ -gulosa)
| | | Receba  $a$  e observe os valores de  $r$  e  $s'$ 
| | |  $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ 
| | até o passo final do episódio ser atingido;
| até o episódio máximo ser atingido;
fim
```

Como já somos capazes de concluir, *Q-learning* precisa armazenar em uma tabela os valores Q incrementais. A demanda computacional para esse armazenamento pode se tornar intratável, conforme crescem os estados e as ações possíveis (Lins, 2020, p. 35).

Outro problema dessa técnica se refere à generalização. A visita aos estados pela escolha ϵ -gulosa não é completa, uma vez que esse algoritmo de busca não o é, conforme vimos quando exploramos o método no tema introdutório a Monte Carlo. Desta forma, podemos não encontrar a política ótima.

Esse problema pode ser contornado pelo uso de aproximadores de função que darão um sentido de busca, orientando a escolha ϵ -gulosa em direção à maximização do resultado. Para tanto, será necessário calcular o gradiente da função Q . A matemática envolvida com derivações parciais é um tanto complexa e não poderemos encetá-la nesse estudo, deixamos a recomendação do estudo da obra de Busoniu *et al.* (2010, p. 43 e seguintes).

Uma alternativa a esse cálculo é a utilização de redes neurais para a estimativa de Q . Essa alternativa, segundo Lins (2020, p. 38), permaneceu de baixa convergência, mas modernas técnicas de aprendizagem em profundidade contornaram o problema da convergência pela análise profunda de parâmetros. O tema de redes neurais será melhor explorado em capítulo próximo, por esse motivo, deixaremos o tema de *Q-learning* nesse ponto.

5.2 SARSA

O método SARSA recebeu esse nome pela justaposição das iniciais *State-Action-Rewards-State-Action*, que designam o procedimento típico de busca da política ótima. Neste caso, a busca ainda se fará pelo valor da ação Q ,



como no algoritmo de *Q-learning*, estudado anteriormente, mas a atualização dos valores de Q não necessita do conhecimento do $Q_{\max}$. Por esse motivo, alguns autores chamam o algoritmo de *Q-learning* de SARSA-Max. Desta forma, a equação iterativa de Q será:

$$Q(s, a) = Q(s, a) + \alpha(r + \gamma Q(s', a') - Q(s, a))$$

O método SARSA segue unicamente o princípio da busca ϵ -gulosa para a escolha de $Q(s', a')$. Quando estudamos *Q-learning*, a escolha do valor de Q do estado atual $Q(s', a')$ era sempre feita por $\max Q(s', a')$. Em SARSA, o algoritmo pode selecionar um Q aleatório qualquer com probabilidade ϵ ou a ação de maior Q com probabilidade $(1 - \epsilon)$.

Você deve estar se perguntando de que forma isto será útil, uma vez que os métodos de controle têm sua existência justificada pela necessidade de encontrarmos os valores ótimos. Podemos direcionar as escolhas do algoritmo para as escolhas de probabilidade majoritária, mas isso reduzirá a excursão da exploração do meio. De qualquer forma a escolha do método de busca gulosa, embora rápido, carrega o estigma da incompletude.

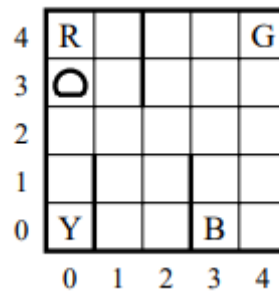
Como pudemos observar, os dois algoritmos de controle *Q-learning* e SARSA permitem que se atualize os valores de estado de forma iterativa e eficiente com a vantagem, em relação à DP clássica e Monte Carlo, de permitir a atualização dos valores durante a operação do agente, sem que se necessite aguardar o atingimento de seu objetivo final. Assume-se o risco de não se obter a melhor política, mas certamente se obterá uma boa política para o agente.

5.3 O Labirinto do Taxista

Uma ótima leitura sobre a comparação entre *Q-learning* e SARSA e sua aplicação em um ambiente de testes hipotético é o artigo de Diettrich (2000). Nesta publicação, o autor apresenta a teoria envolvida em ambos os métodos e propõe um ambiente de teste no qual um pequeno mundo de 5X% é habitado por um motorista de taxi que precisa apanhar seus passageiros e levá-los a um determinado destino. A figura a seguir ilustra o labirinto desse mundo.



Figura 5 – Labirinto do Taxista



Fonte: Diettrich, 2000, p. 236.

Na proposição de Diettrich, o ambiente é episódico e, a cada episódio, o taxista surge em uma posição aleatória. No mesmo episódio, haverá um passageiro em um dos locais R, G, B ou Y escolhido aleatoriamente. Esse passageiro deve ser apanhado pelo taxista e conduzido para o destino, que será escolhido também aleatoriamente em um dos locais R, G, B ou Y.

A cada movimento do taxista, uma recompensa negativa é obtida (-1), da mesma maneira que -10 pontos serão cobrados do motorista se ele executar uma ação ilegal, como deixar o passageiro onde ele não quer. Ao encontrar o destino correto para o passageiro, o taxista obtêm 20 pontos de recompensa. O objetivo é encontrar a política ótima, que maximize a recompensa.

A leitura do artigo nos permite entender como o enfrentamento de um MDP em ambiente episódico pode ser eficientemente, enfrentado por *Q-learning* e SARSA. O ambiente proposto pelo autor se tornou um desafio clássico de RL e é enfrentado recorrentemente pelos desenvolvedores em busca de aperfeiçoar seus algoritmos. Um código que implementa o ambiente do taxista para permitir testes pode ser encontrado na biblioteca OpenAI Gym.

Disponível em: https://github.com/openai/gym/blob/master/gym/envs/toy_text/taxi.py. Acesso em: 25 abr. 2022.

FINALIZANDO

Nesta etapa, entendemos como solucionar um problema de aprendizado em ambiente estocástico, tanto em meios episódicos quanto sequenciais, pelo uso de algoritmos de Monte Carlo e de Diferença Temporal. Até esse ponto,



conseguimos evoluir sem que um conhecimento de IA conexionista fosse necessário. Aprendizagem por reforço é, outrossim, majoritariamente implementada em redes neurais.

Percebemos essa deficiência quando precisamos evoluir o algoritmo de *Q-learning* para além dos limites de baixa dimensionalidade. Por esse motivo, para que concluamos nosso estudo de RL, precisaremos visitar, futuramente, os conceitos de redes neurais, suas arquiteturas e métodos de aprendizagem.



REFERÊNCIAS

- BUSONI, L. et al. **Reinforcement learning and dynamic programming using function approximators**. 2017. CRC PRESS. Disponível em: <<https://orbi.uliege.be/bitstream/2268/27963/1/book-FA-RL-DP.pdf?ref=https://githubhelp.com>>. Acesso em: 22 abr. 2022.
- DIETTERICH, T. G. Hierarchical reinforcement learning with the MAXQ value function decomposition. **Journal of artificial intelligence research**, v. 13, p. 227-303, 2000.
- LINS, R. A. S. **Aprendizagem por reforço profundo uma nova perspectiva sobre o problema dos k-servos**. Tese de Doutorado. UFRN, 2020. Disponível em: <<https://repositorio.ufrn.br/jspui/handle/123456789/29661>>. Acesso em: 22 abr. 2022.
- NORVIG, P. **Inteligência Artificial**. Disponível em: Minha Biblioteca. 3rd. ed. Grupo GEN, 2013. MB
- PELLEGRINI, J.; WAINER, J. Processos de Decisão de Markov: um tutorial. **Revista de Informática Teórica e Aplicada**, v. 14, n. 2, p. 133-179, 2007.
- RAVICHANDIRAN, S. **Hands-on reinforcement learning with Python**: master reinforcement and deep reinforcement learning using OpenAI gym and TensorFlow. Packt Publishing Ltd, 2018.
- SUTTON, R. S. Learning to predict by the methods of temporal differences. **Machine learning**, v. 3, n. 1, p. 9-44, 1988. Disponível em: <<https://link.springer.com/content/pdf/10.1007/BF00115009.pdf>>. Acesso em: 22 abr. 2022.
- SUTTON, R. S.; BARTO, A. G. **Reinforcement learning**: an introduction. MIT press, 2018.